Exercise set 3

Shrinkage methods

Ridge regression and LASSO are called *shrinkage methods* since the regression coeffients are shrunk towards zero.

- OLS: Minimize $RSS(\beta) = \sum_{i=1}^{n}(y_i - \mathbf{x}_i'\beta)^2$
- Ridge regression: Minimize $RSS(\beta) + \lambda \sum_{k=1}^{p} \beta_k^2$
- LASSO: Minimize $RSS(\beta) + \lambda \sum_{k=1}^{p} |\beta_k|$

Task 1

Consider the dataset College. You shall predict the number of applications, `Apps`. Restrict the dataset by

```
College = load_data("College")
xdata = College[["Apps", "Accept", "Top10perc","Outstate","perc.alumni","Expend"]]
xdata.head()
```

See https://islp.readthedocs.io/en/latest/datasets/College.html for a description of the dataset.

a) Split the data into training/test data.

Solution

```
n = len(College)
np.random.seed(123)
ind = np.random.choice(n,size = n//2, replace=False)
mask = np.ones(n,dtype=bool)
mask[ind] = False
train = xdata.iloc[ind]
test = xdata.iloc[mask]
```

b) Compare the estimated coefficients for OLS, ridge regression and LASSO in a regression of `Apps` on `Accept`, `Top10perc` and `Expend`. Try with different values for λ between, 10^{-6} and 10^{6}. Ridge regression and LASSO can be fitted with functions in the `sklearn.linear_model`-library. Example with ridge regression with $\lambda = 1000$

```
import sklearn.linear_model as skm
# Define X and y
Xtrain = train.iloc[:, 1:].values
ytrain = train.iloc[:, 0].values
ridge = skm.Ridge(alpha=100000)
ridge.fit(Xtrain, ytrain)
beta_ridge = ridge.coef_
```

Solution

Example of how to put coefficients in a table.

```
Xtrain_with_const = sm.add_constant(Xtrain)
ols = sm.OLS(ytrain, Xtrain_with_const).fit()
ridge_coef = np.concatenate(([ridge.intercept_], ridge.coef_))
coef_comparison = np.column_stack((ols.params, ridge_coef))
coef_comparison = pd.DataFrame(coef_comparison,columns=['OLS','Ridge'])

print(coef_comparison)
```

```
          OLS       Ridge
0 -552.246063 -640.692659
1    1.345925    1.375640
2   34.401643   13.146620
3   -0.074912   -0.060545
4   -6.531208   -0.287754
5    0.083093    0.116264
```

c) Use the `RidgeCV`- and `LassoCV`-functions to find the optimal λ for both ridge regression and LASSO. Use the argument `cv=5`. Explain what the function is doing and what `cv=5` mean.

Solution

```
import warnings

# Suppress sklearn convergence and runtime warnings
```

```python
warnings.filterwarnings("ignore")

# Define lambda grid
lambdas = np.logspace(-6, 6, 100)

# Ridge CV
ridge_cv = skm.RidgeCV(alphas=lambdas, cv=5)
ridge_cv.fit(Xtrain, ytrain)

lambda_ridge = ridge_cv.alpha_
ridge_opt = skm.Ridge(alpha=lambda_ridge)
ridge_opt.fit(Xtrain, ytrain)

print("Optimal lambda (Ridge):", lambda_ridge)

# Lasso CV
lasso_cv = skm.LassoCV(alphas=lambdas, cv=5, verbose=False, max_iter=10000)
lasso_cv.fit(Xtrain, ytrain)

lambda_lasso = lasso_cv.alpha_
lasso_opt = skm.Lasso(alpha=lambda_lasso)
lasso_opt.fit(Xtrain, ytrain)

print("Optimal lambda (Lasso):", lambda_lasso)
```

```
Optimal lambda (Ridge): 2848.035868435805
Optimal lambda (Lasso): 1e-06
```

cv=5 means that 5-fold cross-validation is used to determine λ.

d) Use the optimal λ's and reestimate the ridge regression and the LASSO. Compare the coefficients with the OLS-estimates.

Solution

```python
ridge_coef = np.concatenate(([ridge_opt.intercept_], ridge_opt.coef_))
lasso_coef = np.concatenate(([lasso_opt.intercept_], lasso_opt.coef_))
coef_comparison = np.column_stack((ols.params, ridge_coef, lasso_coef))
coef_comparison = pd.DataFrame(coef_comparison,columns=['OLS','Ridge','Lasso'])
print(coef_comparison)
```

```
          OLS       Ridge       Lasso
0 -552.246063 -560.804687 -552.246063
1    1.345925    1.348448    1.345925
2   34.401643   32.830520   34.401643
3   -0.074912   -0.074391   -0.074912
4   -6.531208   -5.708781   -6.531208
5    0.083093    0.085417    0.083093
```

e) Evaluate the three predictions by `testMSE` using the valdiation set approach.

Solution

```
ytest = train.iloc[:,0]
Xtest = train.iloc[:,1:]
Xtest_const = sm.add_constant(Xtest)
olspred = ols.predict(Xtest_const)
ridgepred = ridge_opt.predict(Xtest)
lassopred = lasso_opt.predict(Xtest)
MSEols = sum((ytest-olspred)**2)
MSErid = sum((ytest-ridgepred)**2)
MSElas = sum((ytest-lassopred)**2)
print("MSE for OLS: ",round(MSEols,1))
print("MSE for Ridge: ",round(MSErid,1))
print("MSE for Lasso: ",round(MSElas,1))
```

```
MSE for OLS:  442149028.0
MSE for Ridge:  442309293.5
MSE for Lasso:  442149028.0
```

Shrinkage did not help in this case.

Non-linear models

- In *polynomial regression* Y is regressed on $1, X, ..., X^K$.
- In *piecewise constant regression* Y is regressed on indicator variables $I(X < c_1), I(c_1 \leq X < c_2), ..., I(c_{K-1} \leq X < c_K), I(X \geq c_K)$.
- In the *basis function approach* Y is regressed on a number of transformations of X, $b_0(X), b_1(X), b_2(X), ..., b_K(X)$.

- A *cubic spline basis* is defined by

$$\begin{aligned}(b_0(x), b_1(x), b_2(x), b_3(x), b_4(x), ..., b_{K+3}(x)) \\ = (1, x, x^2, x^3, h(x, \xi_1), ..., h(x, \xi_K))\end{aligned}$$

  where

$$h(x, \xi) = (x - \xi)_+^3 = \begin{cases} (x-\xi)^3 & \text{if } x > \xi \\ 0 & \text{otherwise} \end{cases}$$

- In *regression splines* Y is regressed on a cubic spline basis.

Task 2

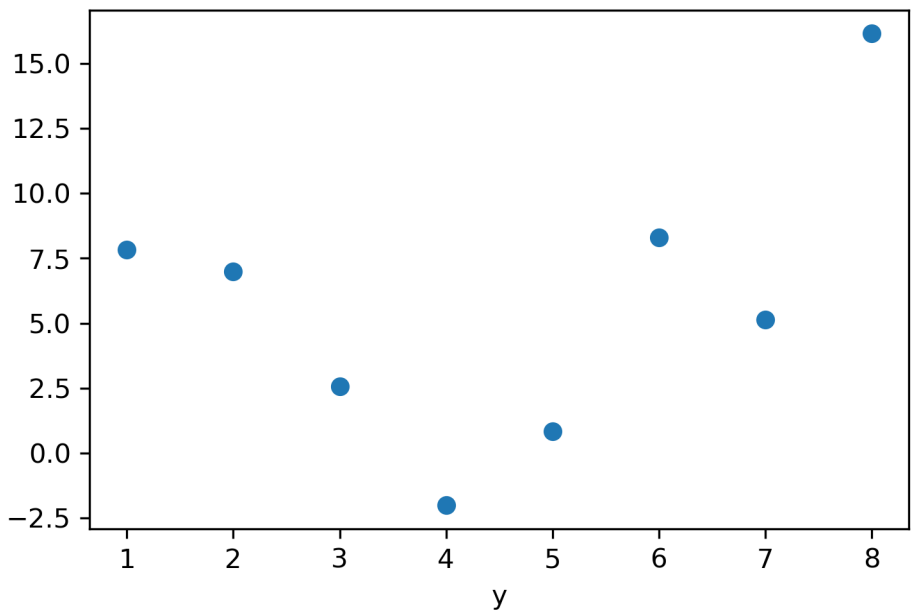
Consider the data set

x	y
1	7.83
2	6.99
3	2.57
4	-2.01
5	0.84
6	8.30
7	5.15
8	16.14

a) Plot y against x.

Solution

```
plt.scatter(df['x'],df['y'])
plt.xlabel('x')
plt.xlabel('y')
plt.show()
```



b) Fit a polynomial regression of order $K = 7$ to the data. Plot the predictions for $x = 1, ..., 8$ on top of the data.

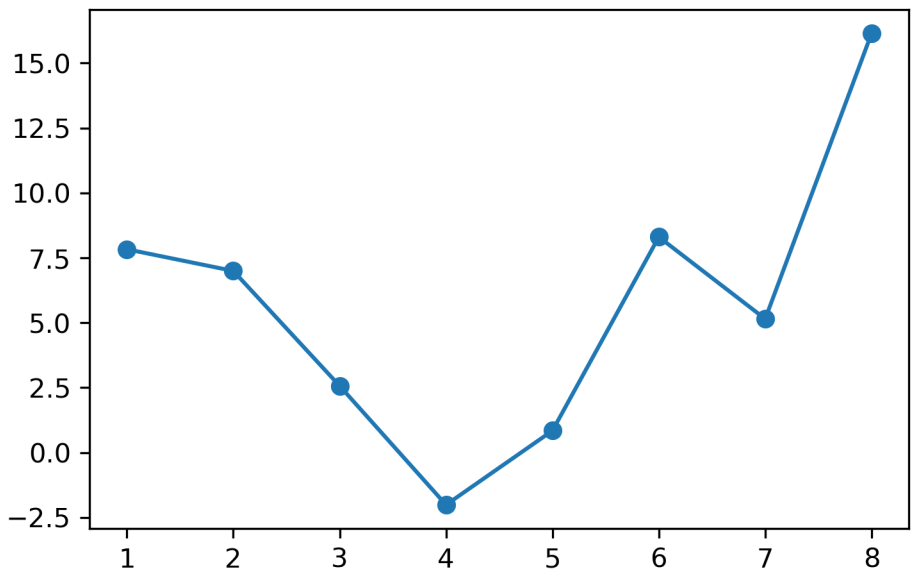
Solution

```
y = df['y']
x = df['x']
X = np.column_stack([x**k for k in range(1,8)])

# Fit linear regression
X_const = sm.add_constant(X)
ols = sm.OLS(y,X_const).fit()

# Predictions
y_pred = ols.predict()

# Plot
plt.scatter(x, y)
plt.plot(x, y_pred)
plt.show()
```



A perfect fit.

c) Fit a regression spline using the `OLS`-function with two knots, $\xi_1 = 3$ and $\xi_2 = 6$. Plot the predictions obtained on top of the data. Do you expect predictions from this model to be better or worse than the prediction in b) for a new x-value? Make sure that you understand what the code is doing.

Solution

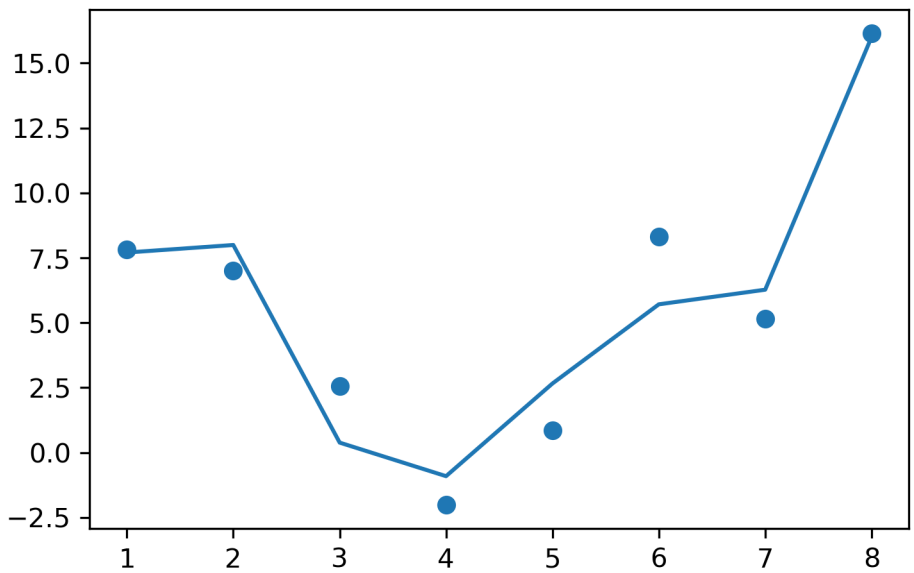
```
def h(x, xi):
    return np.maximum((x - xi)**3, 0)

# Construct X-matrix:
# Intercept, x, x^2, x^3, h(x,3), h(x,6)
X = np.column_stack([
    np.ones_like(x),
    x,
    x**2,
    x**3,
    h(x, 3),
    h(x, 6)
])

# OLS
ols = sm.OLS(y,X).fit()
```

```
# Predictions
y_hat = ols.predict()

# Plot
plt.scatter(x, y)
plt.plot(x, y_hat)
plt.show()
```

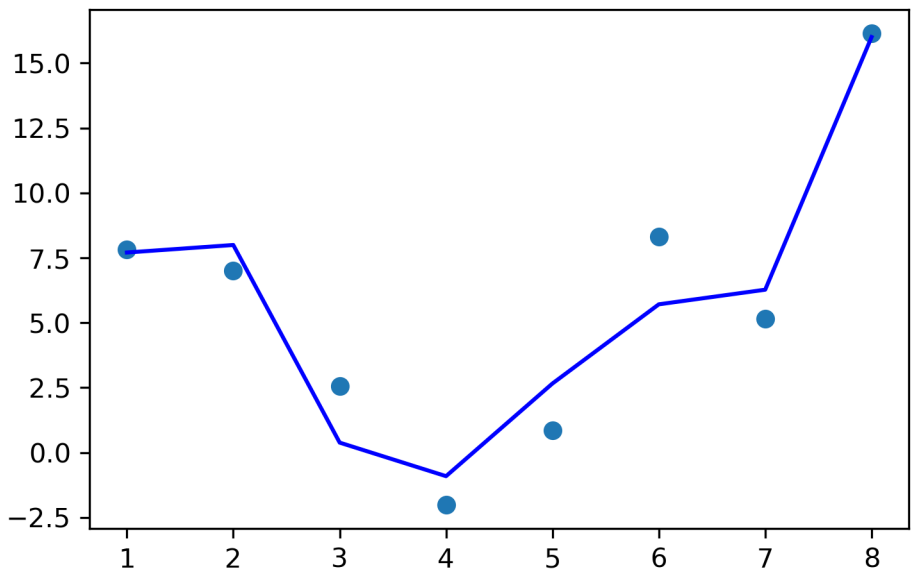


It is difficult to know if the regression spline will do well for a new observation. However the model in c) with a perfect fit seems unlikely to perform well. Imagine, e.g., an observation outside of the interval $[1, 8]$.

d) Use the function `BSpline` in the `ISLP.transforms`-library to produce the same predictions as in c).

Solution

```
from ISLP.transforms import BSpline
knots = [3, 6]
bs = BSpline(internal_knots=knots,intercept=True).fit(x)
bs_x = bs.transform(x)
fit = sm.OLS(y, bs_x).fit()
y_pred2 = fit.predict()
plt.scatter(x,y)
plt.plot(x,y_pred2,color="blue")
```



e) Investigate different placements of the knots and determine which of them are best using the leave-one-out cross-validation.

```
# Truncated power function (same as before)
def h(x, xi):
    return np.maximum((x - xi)**3, 0)

def testMSE(xi1, xi2, x, y):
    n = len(x)
    SE = np.zeros(n)

    for i in range(n):
        # Leave-one-out split
        mask = np.ones(n, dtype=bool)
        mask[i] = False

        x_train = x.drop(i)
        y_train = y.drop(i)
        x_test = x[i]
        y_test = y[i]

        # Design matrix for training data
        X_train = np.column_stack([
            np.ones_like(x_train),
            x_train,
```

```
            x_train**2,
            x_train**3,
            h(x_train, xi1),
            h(x_train, xi2)
        ])

        # OLS estimate
        ols = sm.OLS(y_train,X_train).fit()

        # Design matrix for test point
        X_test = np.column_stack([
            np.ones_like(x_test),
            x_test,
            x_test**2,
            x_test**3,
            h(x_test, xi1),
            h(x_test, xi2)
        ])

        pred = ols.predict(X_test)
        SE[i] = (y_test - pred[0])**2

    return SE.mean()

# Evaluate at different knot choices
print(testMSE(3, 6, x, y))
print(testMSE(2.5, 6.5, x, y))
print(testMSE(3.5, 5.5, x, y))
```

```
3337.4011867738
64099.96265330359
673.9689132724309
```

Among these three alternatives, knots at 3.5 and 5.5 gave the smallest testMSE.